

THE UNIVERSITY OF
MELBOURNE

Final Report

The Art of Scientific Computation Power Laws and Finite Size Scaling in the BTW Abelian Sandpiles Model

Author:
Jonathan Purcell

Subject Handbook code:
COMP90072

Faculty of Science
The University of Melbourne

Draft Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Mathematical Formulation of the BTW Model . . . 2
1.3 Probabilistic Framework . . . 3
1.4 Statistical Models . . . 4

2 Simulation **5**
2.1 Method . . . 5
2.2 Practicalities . . . 5
2.3 Results . . . 6

3 Power Law Fit **7**
3.1 Bounded Power Law Fit . . . 7
3.2 Manually-truncated Power Law . . . 10

4 Finite Size Scaling Effect **12**
4.1 Exponentially Truncated Power Law . . . 12

5 Conclusion **16**
5.1 Reflection . . . 16

Bibliography **16**

1

Introduction

1.1 Motivation

1. Introduce the BTW sandpiles model as a model for self-organised criticality and 1/f noise [1]
2. Define criticality, self-organised criticality, power law scaling, etc

1.2 Mathematical Formulation of the BTW Model

The Abelian Sandpiles Model (or BTW Sandpile Model) was introduced by Bak, Tang, and Wiesenfeld in 1987 [2] as an example of a dynamical system driving itself towards criticality, exhibiting what they call "self-organised criticality".

We introduce the sandpile model, adapting the notation of Járai, 2018 [5].

Define a two-dimensional square lattice with lattice size L as the a graph $G_L = (V_L, E_L)$, where

$$V_L := \{(x, y) \in \mathbb{Z}^2 : 1 \leq x, y \leq L\}$$

defines the set of vertices and

$$E_L := \{\mathbf{xy} : \mathbf{x}, \mathbf{y} \in V_L \wedge \|\mathbf{x} - \mathbf{y}\|_1 = 1\}$$

defines the set of edges, where $\|\mathbf{x} - \mathbf{y}\|_1$ denotes the L_1 norm, i.e. $\|(x_1, x_2) - (y_1, y_2)\|_1 = |x_1 - y_1| + |x_2 - y_2|$.

Let $N(\mathbf{x}) = \{\mathbf{y} \in V_L : \mathbf{xy} \in E_L\}$ be the neighbourhood of $\mathbf{x}$ in G.

A sandpile is a map $\eta : V_L \to \mathbb{Z}_{\geq 0}$, and $\eta(\mathbf{x})$ is called the height at site $\mathbf{x}$.

Define the critical height as $\eta_c := \max_{\mathbf{x} \in V_L} \deg_G(\mathbf{x})$. In the above setup, $\eta_c = 4$ provided $L \geq 3$. A site $\mathbf{x} \in V_L$ is called stable iff $\eta(\mathbf{x}) < \eta_c$ and is otherwise unstable. The sandpile η is stable iff it is stable at each site $\mathbf{x} \in V_L$.

Let $U_\eta = \{\mathbf{x} \in V_L : \eta(\mathbf{x}) \geq \eta_c\}$ be the set of unstable sites.

Let $\Psi_{G_L} = \mathbb{Z}_{\geq 0}^{V_L}$ be the set of possible sandpiles and let $\Omega_{G_L} = \{0, 1, 2, ..., \eta_c - 1\}^{V_L} \subseteq \Psi_{G_L}$ be the set of stable sandpiles.

A toppling is a function $\phi : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ mapping $(\eta, \mathbf{x}) \mapsto \eta'$ where

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} \in N(\mathbf{x}) \wedge \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) - \eta_c & \mathbf{y} = \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) & \text{otherwise} \end{cases}$$

Note that we allow open boundary conditions in this study. When a toppling is initiated at a boundary site, we have $\ell(\eta, \mathbf{x}) := \sum_{\mathbf{x} \in V_L} \eta(\mathbf{x}) - \sum_{\mathbf{x} \in V_L} \eta'(\mathbf{x}) > 0$. This is called the loss associated with toppling η at $\mathbf{x}$.

The avalanche (stabilising) function $\xi : \Psi_{G_L} \to \Omega_{G_L}$ maps $\eta = \eta^{(0)} \mapsto \eta^{(s)}$ where $\eta^{(s)}$ is the stable sandpile output by the following algorithm:

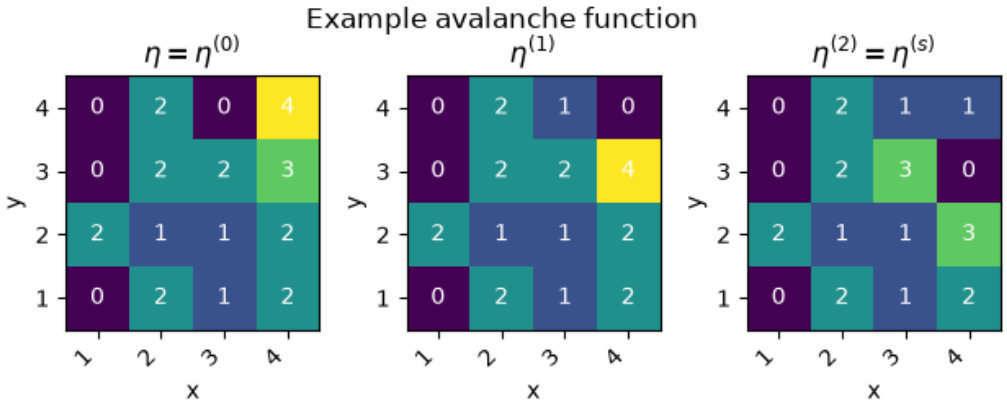


Figure 1.1: Example computation of $\xi(\eta)$ for an unstable sandpile with lattice size $L = 4$ and $\eta_c = 4$. Numbers written at each grid cell represent the sandpile height at vertex (x, y), and correspond to the cell colour. η is first toppled at site $\mathbf{x}^{(1)} = (4, 4)$ to produce $\eta^{(1)}$, which is also unstable. $\eta^{(1)}$ is subsequently toppled at site $\mathbf{x}^{(2)} = (4, 3)$ to produce $\eta^{(2)} \in \Omega_{G_L}$. The algorithm terminates with $s(\eta) = 2$. In the above example, we have $a(\eta) = 2$, $\ell(\eta) = 2 + 1 = 3$, and $d(\eta) = 1$.

Algorithm: Avalanche Function
$i \leftarrow 0$
while $U_{\eta^{(i)}} \neq \emptyset$ **do**:
choose $\mathbf{x}^{(i+1)} \in U_{\eta^{(i)}}$
$\eta^{(i+1)} \leftarrow \phi(\eta^{(i)}, \mathbf{x}^{(i+1)})$
$i \leftarrow i + 1$
end while
return $\eta^{(i)} = \eta^{(s)}$

When $\eta^{(s)} \neq \eta$, we say that an avalanche has occurred.

Avalanches can be described by a range of properties:

1. The avalanche (toppling) size is the final index $s(\eta) = s$
2. The avalanche (toppling) area $a(\eta)$ is the size of the set $\{\mathbf{x}^{(i)} : i \in \{1, \cdots, s\}\}$
3. The loss is $\ell(\eta) := \sum_{i=1}^{s} \ell(\eta^{(i-1)}, \mathbf{x}^{(i)})$
4. The L_1 length is $d(\eta) := \max_{i \in \{1,\cdots,s\}} \left\|\mathbf{x}^{(1)} - \mathbf{x}^{(i)}\right\|_1$.

When an avalanche occurs, note the following constraints:

1. $0 < a(\eta) \leq s(\eta)$
2. $a(\eta) \leq L^2$
3. $0 \leq \ell(\eta) \leq \sum_{\mathbf{x} \in V_L} \eta(\mathbf{x})$
4. $0 \leq d(\eta) \leq 2L$

When no avalanche occurs, we define $s(\eta) = a(\eta) = \ell(\eta) = d(\eta) = 0$ for convenience. However, our focus in this project is on the distribution of these statistics conditional on the occurrence of an avalanche.

A proof that ξ is well-defined and independent of the choice of $\mathbf{x}^{(j)}$ in the loop can be found in Járai, 2018 [5]. The abelian nature of the topplings were first shown by Dhar in 1990 [4].

1.3 Probabilistic Framework

Define the deposition function $\delta : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ as the map $\eta \mapsto \eta'$ with rule

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} = \mathbf{x} \\ \eta(\mathbf{y}) & \mathbf{y} \neq \mathbf{x} \end{cases}$$

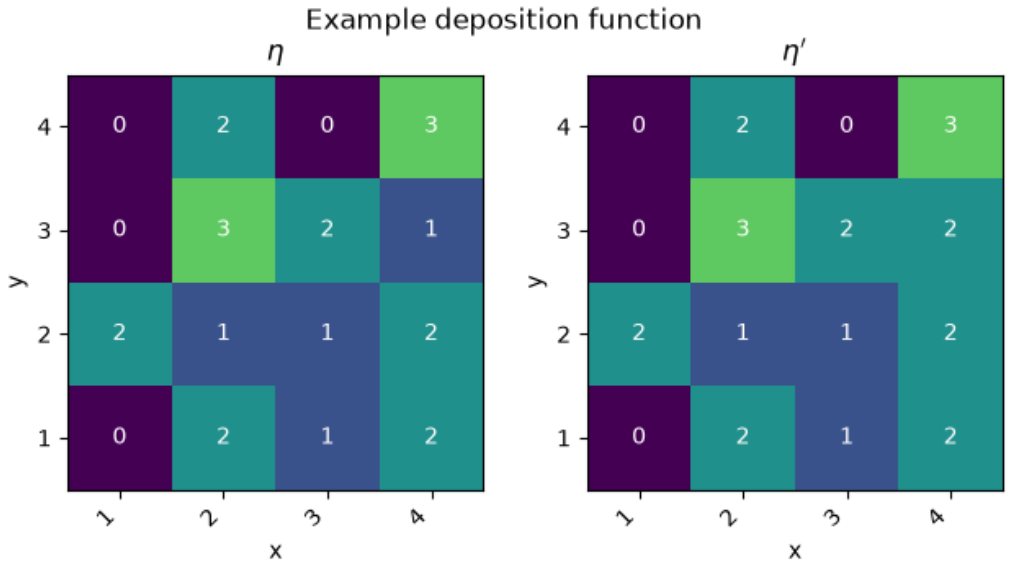


Figure 1.2: Example deposition function δ applied to a sandpile with lattice size $L = 4$. Numbers written at each grid cell represent the sandpile height at vertex (x, y), and correspond to the cell colour. A grain is deposited at site $\mathbf{x} = (4, 3)$, increasing the site height from 1 to 2 and producing η'.

We describe the sandpile evolution process by considering the sequence of iid random variables $\mathbf{X}_1, \mathbf{X}_2 ...$, each with support V_L and pmf

$$p(\mathbf{X}_j = \mathbf{x}) = L^{-2}$$

This represents a sequence of uniformly chosen random vertices, which are taken as locations for grain deposition.

From the sequence $\{X_j\}_{j\geq 1}$, we can build a sequence of stable sandpiles $\{\eta_j\}_{j\geq 0}$ with $\eta_0 = \mathbf{0}$ and $\eta_j = \xi(\delta(\eta_{j-1}, \mathbf{X}_j))$ for $j \geq 1$.

Likewise, we obtain the sequence of avalanche sizes $\{S_j\}_{j\geq 1}$ where $S_j := s(\delta(\eta_{j-1}, \mathbf{X}_j))$. A similar construction yields sequences of avalanche areas, losses, and L_1 lengths.

The sequence $\{\eta_j\}_{j\geq 0}$ forms a discrete Markov Chain with finite state space Ω_{G_L}; the distribution of η_{j+1} depends only on the previous state η_j and the transition probability is determined by the distribution of $\mathbf{X}_j$.

Jarai [5] discusses the Markov structure in detail, and proves that the distribution of states η_j converges to a stationary distribution $\pi(\eta)$ as $j \to \infty$, where $\pi(\eta) > 0$ iff η is a recurrent state in the Markov Chain.

Consequently, the sequence of avalanche sizes $\{S_j\}_{j\geq 1}$ and the other avalanche statistic sequences, which are functions of the states η_j, also converge in distribution to stationary distributions as $j \to \infty$.

1.4 Statistical Models

Describe modelling choices for power law and truncated power law distributions

2

Simulation

2.1 Method

The stationarity of the avalanche statistic sequences forms the theoretical foundation for empirical simulation and analysis according to the following algorithm:

Algorithm: Simulating the Sandpile Model
initialise $\mathbf{s}, \mathbf{a}, \ell, \mathbf{d}$ as empty arrays of size n
initialise $\eta = \mathbf{0}$
for $i \in \{1, \cdots, m\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$\eta \leftarrow \xi(\eta)$
end for
for $i \in \{m+1, \cdots, m+n\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$\mathbf{s}[i-m] \leftarrow s(\eta)$
$\mathbf{a}[i-m] \leftarrow a(\eta)$
$\ell[i-m] \leftarrow \ell(\eta)$
$\mathbf{d}[i-m] \leftarrow d(\eta)$
$\eta \leftarrow \xi(\eta)$
end for
return $\mathbf{s}, \mathbf{a}, \ell, \mathbf{d}$

We can then discard the zero values of the returned arrays, which are not associated with an avalanche.

In the above, simulated states $\eta_1, \cdots, \eta_m$ are called burn-in states, and are discarded. After this period, we assume that the distribution of η_j is approximately stationary. Therefore, simulations $\eta_{m+1}, \cdots, \eta_{m+n}$ are retained and their empirical distribution approximates the stationary distribution $\pi(\eta)$. From that sequence, we construct simulated sequences for each avalanche statistic of length n, whose distribution approximates its respective stationary distribution.

2.2 Practicalities

In practice, hundreds of thousands of simulations are required to approach stationarity. This is computationally expensive, especially for large lattice sizes, because computations cannot be parallelised due to the Markov structure of the problem. An initial Python implementation of the simulation procedure above was too slow to produce meaningful results. The final implementation uses `numba` with `njit` decorators, which stands for 'no python just in time' [6]. This compiles the Python code prior to execution, bypassing the CPython interpreter and achieving execution speed near code written in C. However, only a subset of Python and `numpy` functionality is permitted, requiring significant refactoring away from object-oriented design.

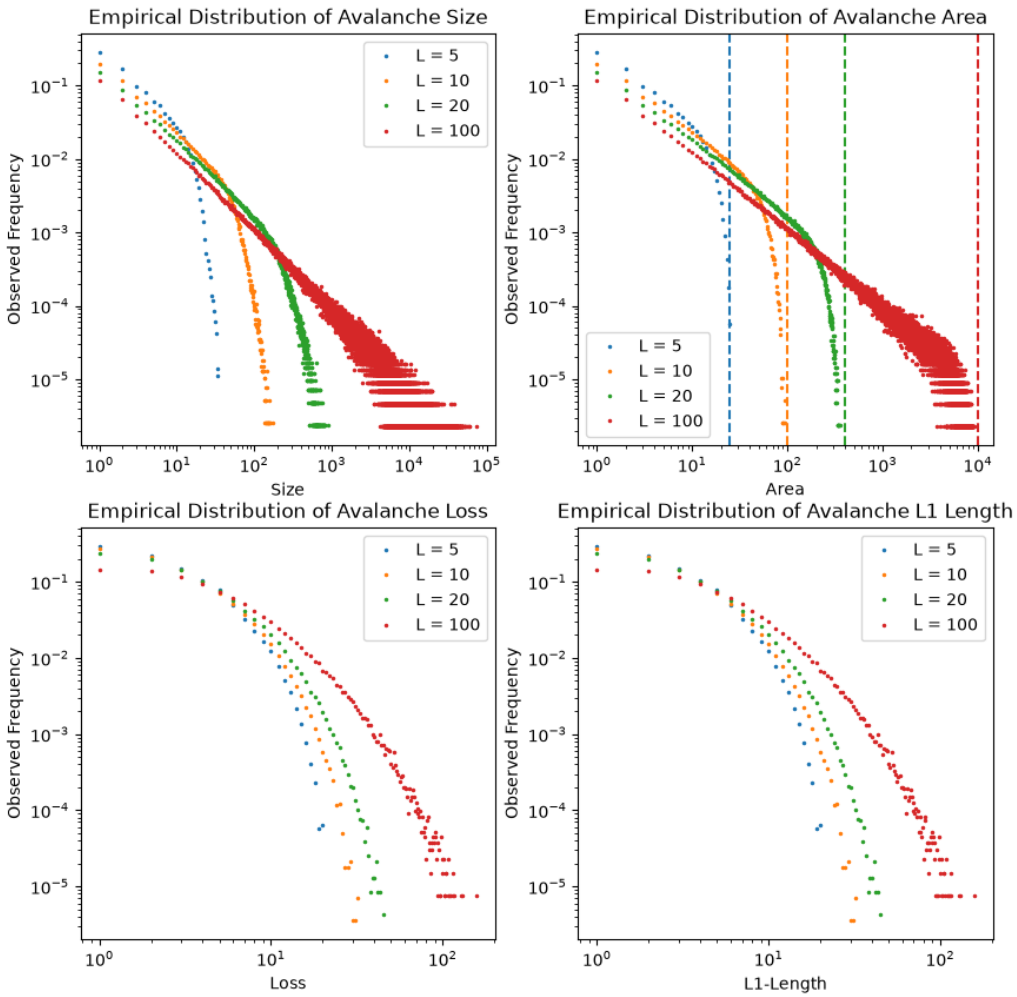


Figure 2.1: Observed frequencies of key avalanche statistics computed from the simulated sandpile Markov chain, excluding the discarded burn-in samples. A subset of four lattice sizes are depicted: 5, 10, 20, and 100, with varying colors. Values are computed from 10^6 simulated observations. The dashed vertical lines on the top right correspond to the maximum possible avalanche area for each lattice size, L^2. Plots are shown in log-log axes. The appearance of horizontal lines is an artifact of the log-log axes. The lines correspond to sizes with only 1, 2, 3, etc. observations in the dataset, with exaggerated vertical distance between the lines because of the log scale.

2.3 Results

Simulations were performed for lattice sizes $L \in \{5, 10, 20, 30, 40, 50, 65, 80, 100\}$. The burn-in period was set to $m = 4 \times 10^4$ and sequences of length $n = 1 \times 10^6$ were retained for each lattice size.

The plots in Figure 2.1 depect a linear trend in early values of avalanche size and avalanche area, suggesting a power law distribution followed by a sharp decay. The distributions for Loss and L1 Length do not appear to have this power law structure, and the remainder of this report focusses on understanding and analysing the distribution of avalanche size and area.

3

Power Law Fit

3.1 Bounded Power Law Fit

The linear component of the distribution observed in Figure 2.1 suggests fitting a power law distribution to the simulated sequence of avalanche sizes and areas.

We hypothesize a power law scaling relationship for Toppling Size S and area A of the form:

$$p(X = x; \tau, x_{max}) = C_{\tau, x_{max}} x^{-\tau} \text{ for } x \in \{1, \cdots, x_{max}\}$$

where $x_{max} = x_{max}(L)$ is a cutoff value reflecting the observed decay of the distribution away from power law scaling due to the finite lattice size L.

The shape of the pmf under varying parameter values can be observed in Figure 3.1.

To estimate the power law scaling coefficient τ, we apply the method of Maximum Likelihood as justified in Section 1.4.

Let $\mathbf{x} = (x_1, x_2, \cdots, x_n)$. The likelihood function is:

$$\mathcal{L}(\mathbf{x}; \tau, x_{max}) = \prod_{i=1}^{n} \frac{x_i^{-\tau}}{C_{\tau, x_{max}}}$$

$$\log \mathcal{L}(\mathbf{x}; \tau, x_{max}) = -n \log C_{\tau, x_{max}} - \tau \sum_{i=1}^{n} \log x_i$$

Where $C_{\tau, x_{max}}$ is the normalising constant:

$$C_{\tau, x_{max}} = \sum_{k=1}^{x_{max}} k^{-\tau}$$

To estimate the parameter τ assuming known x_{max}, we compute its Maximum Likelihood Estimate (MLE). This is the value $\hat{\tau}$ that satisfies:

$$\hat{\tau} = \arg\max_{\tau} \mathcal{L}(\mathbf{x}; \tau, x_{max}) = \arg\min_{\tau} \{-\log \mathcal{L}(\mathbf{x}; \tau, x_{max})\}$$

Because the normalising constant $C_{\tau, x_{max}}$ depends on the parameter τ, we cannot find an analytic solution for $\hat{\tau}$ and rely on numerical optimisation techniques. The numerical optimisation technique used here is the L-BFGS-B algorithm introduced by Byrd et al. (1995) [3] and implemented in the `scipy` python library.

Let $\tau = \tau_s(L), x_{max} = s_{max}(L)$ for modelling the avalanche size, and $\tau = \tau_a(L), x_{max} = a_{max}(L)$ for modelling the avalanche area. We first consider a full-data estimation procedure, where we ignore the non-linear distribution tails and fit $\tau_s(L)$ and $\tau_a(L)$ for each lattice size simulated. Here, we fix $a_{max}(L) = L^2$ and $s_{max}(L) = L^3$ as known; this is clearly justified as the theoretical maximum for avalanche area, and is a computational adjustment for avalanche size that exceeds any observed avalanche size in the simulated data.

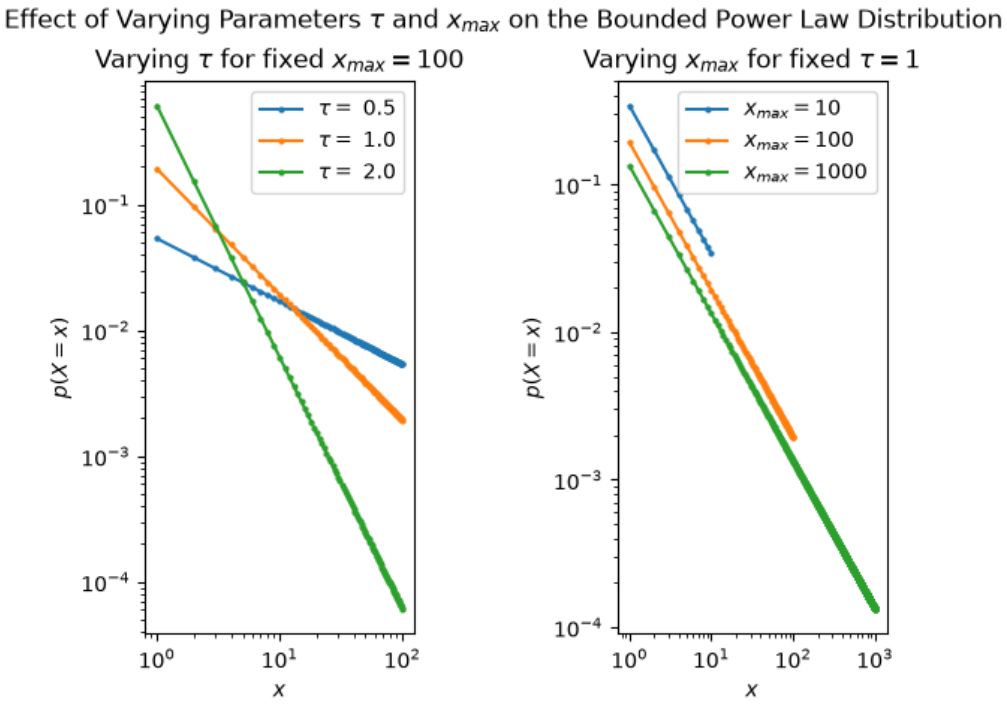


Figure 3.1: Visualisation of the bounded power law probability mass function on log-log axes under different parameter combinations. Each point represents a discrete value and corresponding probability mass, with a line connecting sequential points for illustration purposes. The left plot shows the effect of varying τ while keeping x_{max} constant, which results in changes to the slope and intercept of the connecting line. The right plot shows the effect of varying x_{max} while keeping τ constant, which results in vertical translation of the connecting line, with different support for x. The orange line is identical in both plots, corresponding to parameters $\tau = 1$, $x_{max} = 100$.

Under the finite size scaling hypothesis discussed in 1.4, we expect that the power law coefficient should be constant for all values of L, i.e. we expect $\tau_s(L) = \tau_s$ and $\tau_a(L) = \tau_a$.

The estimates obtained are presented in Figure 3.2 and Table 3.1. We observe a decreasing trend in the estimated power law coefficients for both avalanche size and avalanche area. However, in Figure 3.3 we see that the tail has a substantial effect on the fit, creating an artificially high estimate for the slope. This effect appears most pronounced at smaller lattice sizes, but is still evident at $L = 100$, suggesting the decrease in $\hat{\tau}$ could be caused by poor fit at the tails.

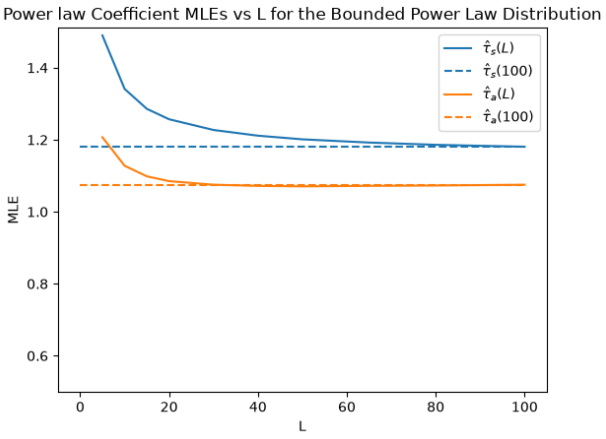


Figure 3.2: Estimates for $\hat{\tau}_s(L), \hat{\tau}_a(L)$. Dashed horizontal lines reflect the estimates $\hat{\tau}_s(100)$ and $\hat{\tau}_a(100)$.

L	$\hat{\tau}_s(L)$	$\hat{\tau}_a(L)$
5	1.49	1.21
10	1.34	1.13
15	1.29	1.10
20	1.26	1.09
30	1.23	1.08
40	1.21	1.07
50	1.20	1.07
65	1.19	1.07
80	1.19	1.07
100	1.18	1.08

Table 3.1: MLE power-law slopes obtained for varying lattice sizes L for the bounded power law distribution

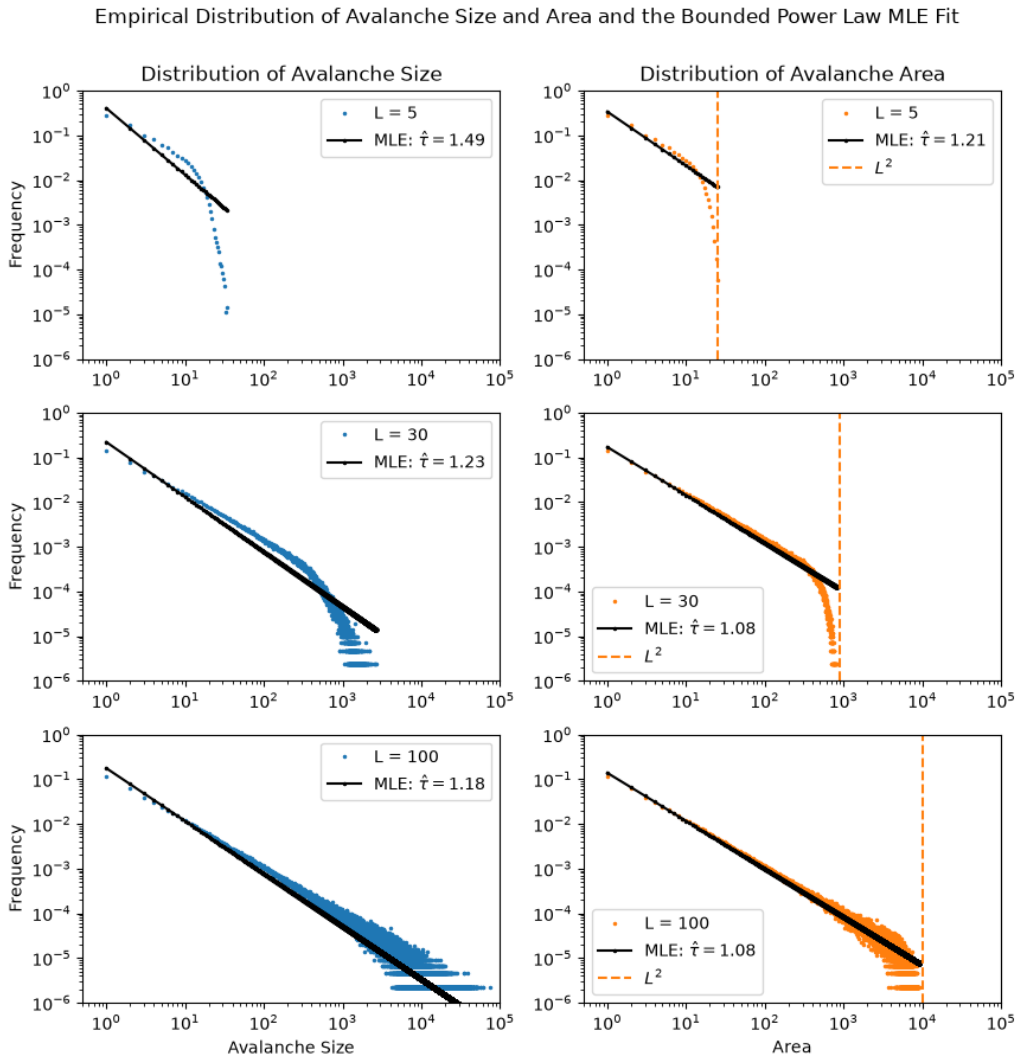


Figure 3.3: Fitted MLEs for a power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure **??**. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The right column shows avalanche size, while the left column shows avalanche area. The dashed vertical lines on the left show the bound on avalanche area, L^2.

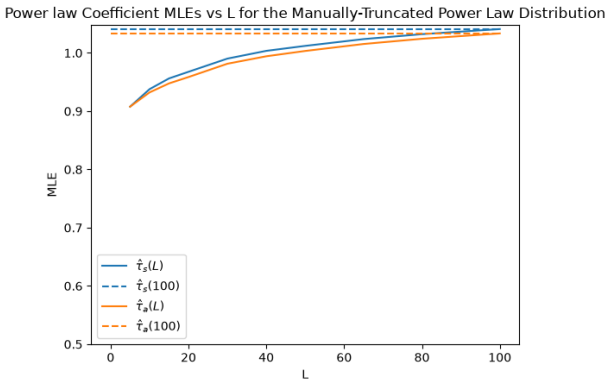


Figure 3.4: Estimates for $\hat{\tau}_s(L), \hat{\tau}_a(L)$ under the manual truncation rule $s_{max} = a_{max} = L^2/10$. Dashed horizontal lines reflect the estimates $\hat{\tau}_s(100)$ and $\hat{\tau}_a(100)$.

L	$\hat{\tau}_s(L)$	$\hat{\tau}_a(L)$
5	0.91	0.91
10	0.94	0.93
15	0.96	0.95
20	0.97	0.96
30	0.99	0.98
40	1.00	0.99
50	1.01	1.00
65	1.02	1.02
80	1.03	1.02
100	1.04	1.03

Table 3.2: MLE power-law slopes obtained for varying lattice sizes L under the manual truncation rule $s_{max} = a_{max} = L^2/10$

3.2 Manually-truncated Power Law

As a first correction for the finite lattice effect, we consider restricting s_{max} and a_{max} to include only those topple sizes and areas for which linearity is observed in the log-log plot. As we saw in Figure **??**, changing x_{max} should not affect the estimated slope of the distribution.

To avoid manually setting an arbitrary maximum for each value of L, here we consider the rule-of-thumb approximation $s_{max} = a_{max} = 0.1L^2$. Clearly, there are significant limitations to this approach: the cutoff rule was chosen arbitrarily, and data is significantly limited for lower values of L. However this will help us get a more accurate idea of the power law structure.

The results in Table 3.2 and Figure 3.4 are noticeably lower than the previous estimates in Table 3.1, with an increasing trend rather than decreasing. From these two analyses, it is reasonable that the powerlaw-governed section of the distribution has exponent $\tau_s, \tau_a \approx 1$, which is consistent with the estimate presented in the original 1987 paper by Bak et al. [1].

To avoid discarding data, we turn to explicit models of the tail distribution, which we use to attempt to capture the effect of the finite lattice size L and obtain a more informed approximation to the powerlaw scaling coefficient.

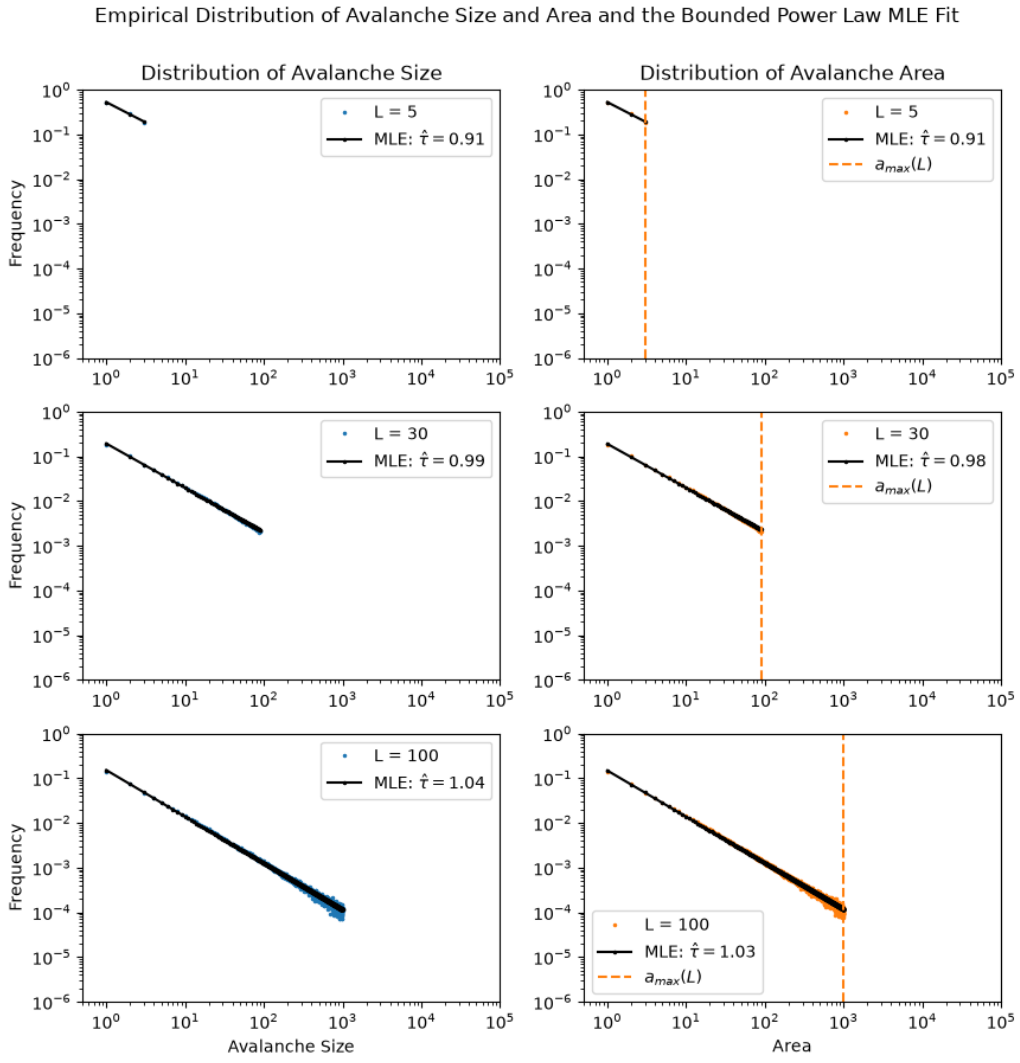


Figure 3.5: Fitted MLEs for the manually-truncated power law distribution against the simulated data, for select lattice sizes $L = 5, 30, 100$. The full data fits are depicted in Appendix Figure **??**. MLEs are drawn as black points with a connecting line drawn for illustration purposes. The right column shows avalanche size, while the left column shows avalanche area. The dashed vertical lines on the left show the manual bound on avalanche area, $L^2/10$.

4

Finite Size Scaling Effect

Recalling Section 1.4, the finite size scaling law hypothesizes that the tail behaviour relates to the finite lattice size L via a probability mass function of the form:

$$p(X = x; \tau, L, f, D) \propto x^{-\tau} f\left(\frac{x}{L^D}\right)$$

Here f governs the tail behaviour of the distribution, i.e. for small values of u, $f(u) \approx 1$ and the distribution resembles a power law.

4.1 Exponentially Truncated Power Law

We first consider the exponentially-truncated power law:

$$p(X = x; \tau, c, x_{max}) = \frac{1}{C_{\tau,c,x_{max}}} x^{-\tau} \exp\left(\frac{-x}{c}\right)$$

Where $C_{\tau,c,x_{max}}$ is the normalising constant:

$$C_{\tau,c,x_{max}} = \sum_{k=1}^{x_{max}} k^{-\tau} \exp\left(-k/c\right)$$

In the above pmf, τ and x_{max} are, as before, the power law scaling coefficient and the support upper bound respectively, while the new parameter c controls the location of the soft cutoff, where power law behaviour decays.

The shape of the pmf under varying parameter values can be observed in Figure 4.1

The likelihood and log-likelihood are:

$$\mathcal{L}(\mathbf{x}; \tau, c, x_{max}) = \prod_{i=1}^{n} \frac{1}{C_{\tau,c,x_{max}}} x_i^{-\tau} \exp\left(\frac{-x_i}{c}\right)$$

$$\log \mathcal{L}(\mathbf{x}; \tau, c, x_{max}) = -n \log C_{\tau,c,x_{max}} - \tau \sum_{i=1}^{T} \log x_i - \frac{1}{c} \sum_{i=1}^{T} x_i$$

In the above, we model $f(u) = \exp(-u)$. We expect $c_a \propto L^D$ for some constant D across all lattice sizes. The advantage of this structure is that the full (not truncated) dataset can be used to fit the powerlaw exponent τ_a, with c_a representing a soft cutoff boundary that is fit to the data, rather than a hard cutoff that is chosen arbitrarily. If the distribution obeys finite size scaling with this f, then we should expect to see a relatively constant τ_a across varying L, and we should observe a linear $\log(c_a)$ vs $\log(L)$ plot with constant slope D_a. In particular, we can estimate pooled MLEs from the full dataset across different values of L according to

$$p(a; L, \tau_a, \gamma_a, D_a) = \frac{1}{C_{L,\tau_a,\gamma_a,D_a}} a^{-\tau_a} \exp\left(\frac{a}{\gamma_a L^{D_a}}\right)$$

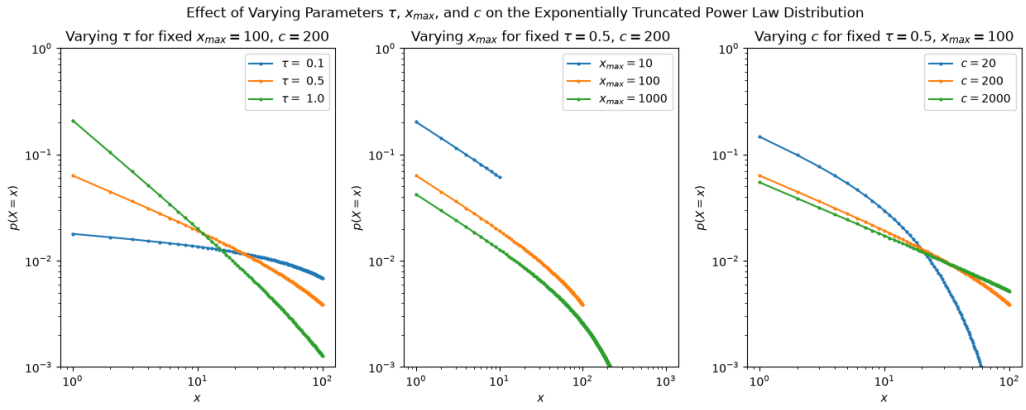


Figure 4.1: Visualisation of the exponentially truncated power law probability mass function on log-log axes under different parameter combinations. Each point represents a discrete value and corresponding probability mass, with a line connecting sequential points for illustration purposes. The left plot shows the effect of varying τ while keeping x_{max} and c constant, which results in changes to the slope of the linear part of the connecting line. The central plot shows the effect of varying x_{max} while keeping τ and c constant, which results in vertical translation of the connecting curve, with different support for x. The right plot shows the effect of increasing c while keeping τ and x_{max} constant, which results in later decay.The orange line is identical in all plots, corresponding to parameters $\tau = 0.5$, $x_{max} = 100$, and $c = 200$.

Denote $\mathbf{a}_L$ the set of observations generated from the lattice of size L. Then likelihood and log-likelihood in the pooled case are:

$$\mathcal{L}(\mathbf{a}_{L_1}, \cdots \mathbf{a}_{L_K}; \tau_a, \gamma_a, D_a) = \prod_{j=1}^{K} \mathcal{L}(\mathbf{a}_{L_j}; \tau_a, c_a = \gamma_a L_j^{D_a}, a_{max} = L^2)$$

We follow the below procedure:

1. For each L, estimate MLE parameters $\hat{\tau}_a(L), \hat{c}_a(L)$.
2. Examine the MLE fit against the observed data
3. Pool the data and estimate MLE parameters $\tau_a, gam\hat{m}a_a, \hat{D}_a$.
4. Examine the MLE fit of $\log \hat{\gamma}_a, \hat{D}_a$ against $\log \hat{c}_a(L)$ and $\log L$.

4.1.1 Estimating MLE parameters for each L

As before, for each L, we numerically optimise the log-likelihood function to obtain:

$$\hat{\tau}_a(L), \hat{c}_a(L) = \underset{\tau_a, c_a}{\arg\min} \left\{ -\log \mathcal{L}(\mathbf{a}_L; \tau_a, c_a, a_{max} = L^2) \right\}$$

The results in Table 4.1 are promising: for lattice sizes 40 and above, the powerlaw coefficient $\tau \approx 1$. Further, the soft cutoffs c appear to scale appropriately with L, a relationship that we will examine in more detail when computing the pooled estimators.

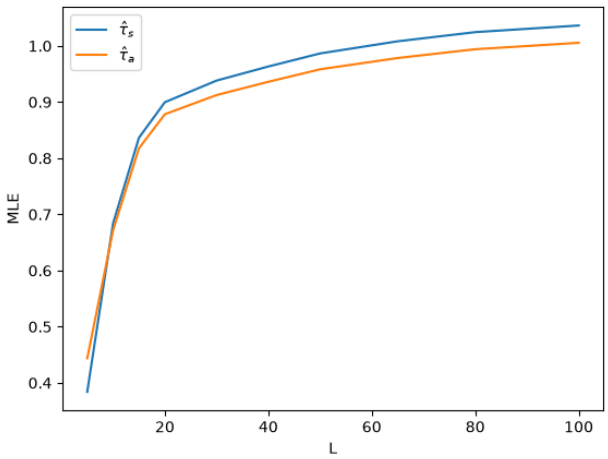


Figure 4.2: Estimates MLEs for τ_s, τ_a for varying lattice sizes L.

L	τ_s	c_s	τ_a	c_a
5	0.38	5.69	0.44	6.27
10	0.68	27.49	0.67	26.58
15	0.84	81.55	0.82	74.76
20	0.90	169.69	0.88	148.70
30	0.94	444.13	0.91	355.15
40	0.96	902.92	0.94	671.42
50	0.99	1623.38	0.96	1124.59
65	1.01	3237.96	0.98	2020.08
80	1.02	5846.02	0.99	3306.74
100	1.04	10599.48	1.01	5372.85

Table 4.1: MLE power-law slopes τ and soft cutoffs c obtained for varying lattice sizes L

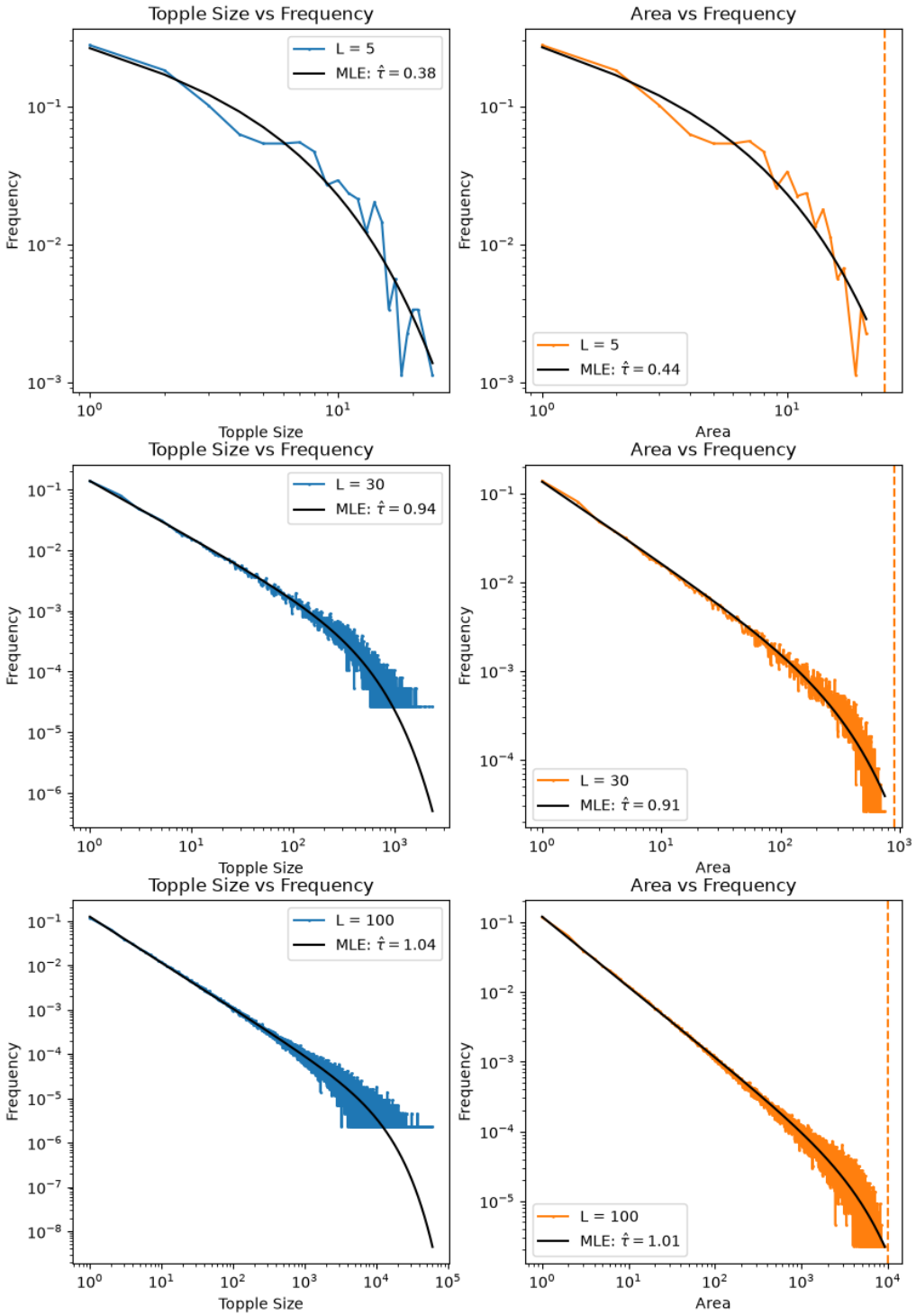


Figure 4.3: Fitted MLEs for the exponentially-truncated power law distribution against the observed data.

5

Conclusion

5.1 Reflection

Evidence of problem solving and creativity

Evidence that student has extended their knowledge

Shows initiative

Bibliography

[1] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Physical review letters*, 59(4):381, 1987.

[2] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Phys. Rev. Lett.*, 59:381–384, Jul 1987.

[3] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing*, 16(5):1190–1208, 1995.

[4] D. Dhar. Self-organized critical state of sandpile automaton models. *Physical review letters*, 64(14):1613, 1990.

[5] A. A. Járai. Sandpile models. *Probability Surveys*, 15(none):243 – 306, 2018.

[6] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, pages 7:1–7:6, New York, NY, USA, 2015. ACM.